Experiment 10: Implementation of Autoencoders for Data Denoising using Cardiovascular Dataset

1. Dataset Source

The dataset used for this experiment is the **Cardiovascular Disease Dataset**.

- Source: Kaggle
- Dataset Name: Cardiovascular Disease Dataset (`cardio_train.csv`)
- The dataset contains real-world medical data for predicting heart disease.

2. Dataset Description

The dataset consists of patient health records used to analyze cardiovascular conditions.

Features:

Feature	Description
age	Age of patient (in days, converted to years)
gender	Gender
height	Height (cm)
weight	Weight (kg)
ap_hi	Systolic blood pressure
ap_lo	Diastolic blood pressure
cholesterol	Cholesterol level
gluc	Glucose level

smoke	Smoking habit
alco	Alcohol consumption
active	Physical activity
cardio	Target variable

Dataset Characteristics:

- Number of records: ~70,000
- Number of features: 11
- Type: Structured tabular dataset
- Problem Type: Feature reconstruction and denoising

3. Mathematical Formulation of Autoencoder

An Autoencoder is an unsupervised neural network used to learn compressed representations of data.

Encoder Function:

$$h = f(x) = \sigma(Wx + b)$$

Decoder Function:

$$\hat{x} = g(h) = \sigma(W'h + b')$$

Loss Function (Reconstruction Error):

$$L = \frac{1}{n} \sum (x - \hat{x})^2$$

Where:

- (x) = original input
- (\hat{x}) = reconstructed output

4. Algorithm Limitations

- Not originally designed for tabular data
- Requires proper scaling of features
- Sensitive to noise level
- May reconstruct noise if not trained properly
- Requires careful tuning of architecture

5. Methodology / Workflow

The following steps were performed:

1. Load cardiovascular dataset
2. Data preprocessing and feature scaling
3. Add synthetic noise to dataset
4. Split into training and testing sets
5. Build Autoencoder model (encoder + decoder)
6. Train model using noisy input and clean output
7. Reconstruct denoised data
8. Evaluate reconstruction error
9. Visualize results

Workflow Diagram:

Dataset → Scaling → Add Noise → Autoencoder Model
→ Training → Reconstruction → Evaluation → Visualization

6. Performance Analysis

The model performance was evaluated using:

- Reconstruction Loss (MSE)
- Loss Curve (training vs validation)
- Comparison of noisy vs reconstructed data
- Error distribution

Observations:

- The autoencoder successfully reduced noise in the data
- Loss decreased steadily during training
- Reconstructed data closely matched original data
- Error distribution indicated effective denoising

7. Hyperparameter Tuning

The following parameters were tuned:

- Number of hidden layers
- Number of neurons in encoder/decoder
- Learning rate
- Batch size
- Number of epochs

Impact:

- Improved reconstruction quality
- Reduced error
- Better convergence
- Enhanced feature representation

CODE:

```
# ==============================

# IMPORT LIBRARIES

# ==============================

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split


from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, Dense


# ==============================

# LOAD DATA

# ==============================

df = pd.read_csv('/content/cardio_train.csv', sep=';')


df = df.drop('id', axis=1)

df['age'] = df['age'] / 365


X = df.drop('cardio', axis=1)
```

```
# ==============================
# SCALE DATA
# ==============================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ==============================
# 🔥 ADD NOISE (IMPORTANT)
# ==============================
noise_factor = 0.3
X_noisy = X_scaled + noise_factor * np.random.normal(size=X_scaled.shape)

# ==============================
# TRAIN TEST SPLIT
# ==============================
X_train, X_test = train_test_split(X_scaled, test_size=0.2, random_state=42)
X_train_noisy, X_test_noisy = train_test_split(X_noisy, test_size=0.2, random_state=42)

# ==============================
# AUTOENCODER MODEL
# ==============================
input_dim = X_train.shape[1]

input_layer = Input(shape=(input_dim,))
```

```
# Encoder
encoded = Dense(16, activation='relu')(input_layer)
encoded = Dense(8, activation='relu')(encoded)

# Decoder
decoded = Dense(16, activation='relu')(encoded)
decoded = Dense(input_dim, activation='linear')(decoded)

# Model
autoencoder = Model(input_layer, decoded)

autoencoder.compile(optimizer='adam', loss='mse')

autoencoder.summary()

# ==============================
# TRAIN MODEL
# ==============================
history = autoencoder.fit(
    X_train_noisy, X_train,
    epochs=30,
    batch_size=32,
     validation_data=(X_test_noisy, X_test)
)

# ==============================
```

```
# 📈 LOSS GRAPH
# ==============================
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Autoencoder Loss")
plt.legend(['Train','Validation'])
plt.show()

# ==============================
# 🔥 DENOISING RESULT
# ==============================
decoded_data = autoencoder.predict(X_test_noisy)

# ==============================
# 🎨 VISUALIZATION (IMPORTANT)
# ==============================

# Compare noisy vs reconstructed
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.plot(X_test_noisy[0])
plt.title("Noisy Data")

plt.subplot(1,3,2)
plt.plot(decoded_data[0])
```

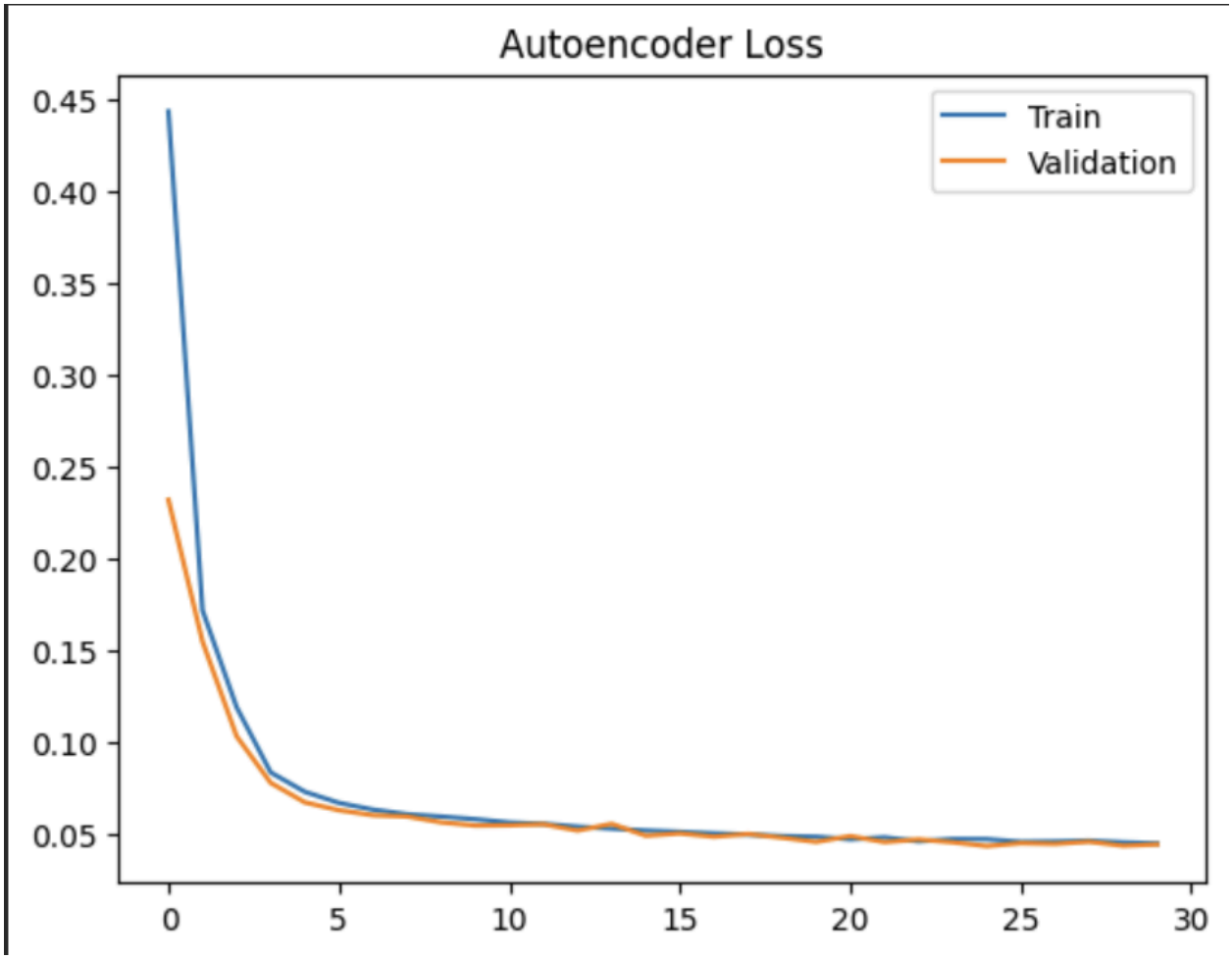
```
plt.title("Denoised Output")


plt.subplot(1,3,3)
plt.plot(X_test[0])
plt.title("Original Data")


plt.show()


# ==============================
# ERROR DISTRIBUTION
# ==============================
error = np.mean(np.power(X_test - decoded_data, 2), axis=1)


sns.histplot(error, bins=50)
plt.title("Reconstruction Error")
plt.show()
```

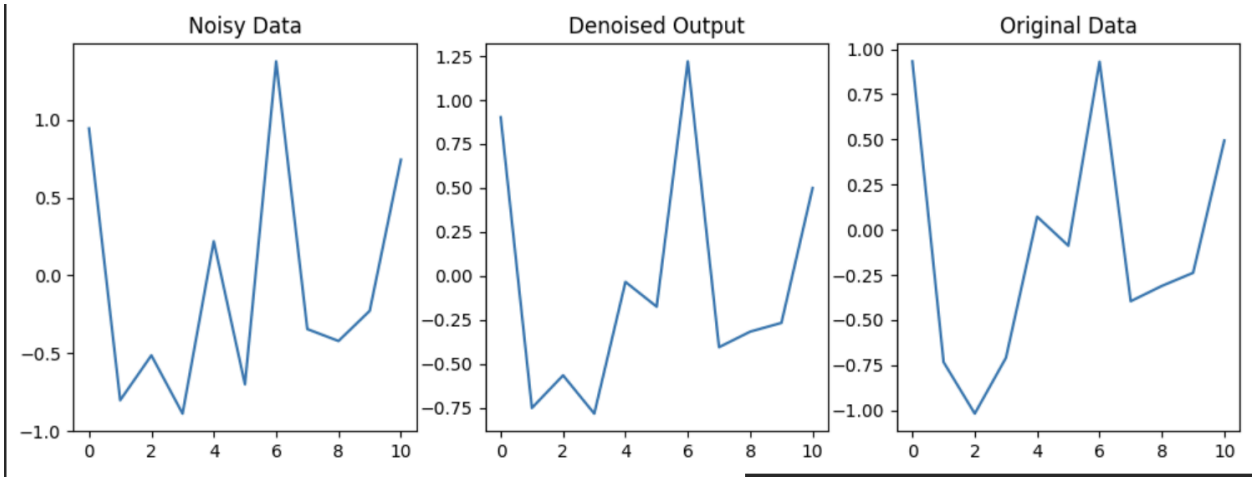
OUTPUT:

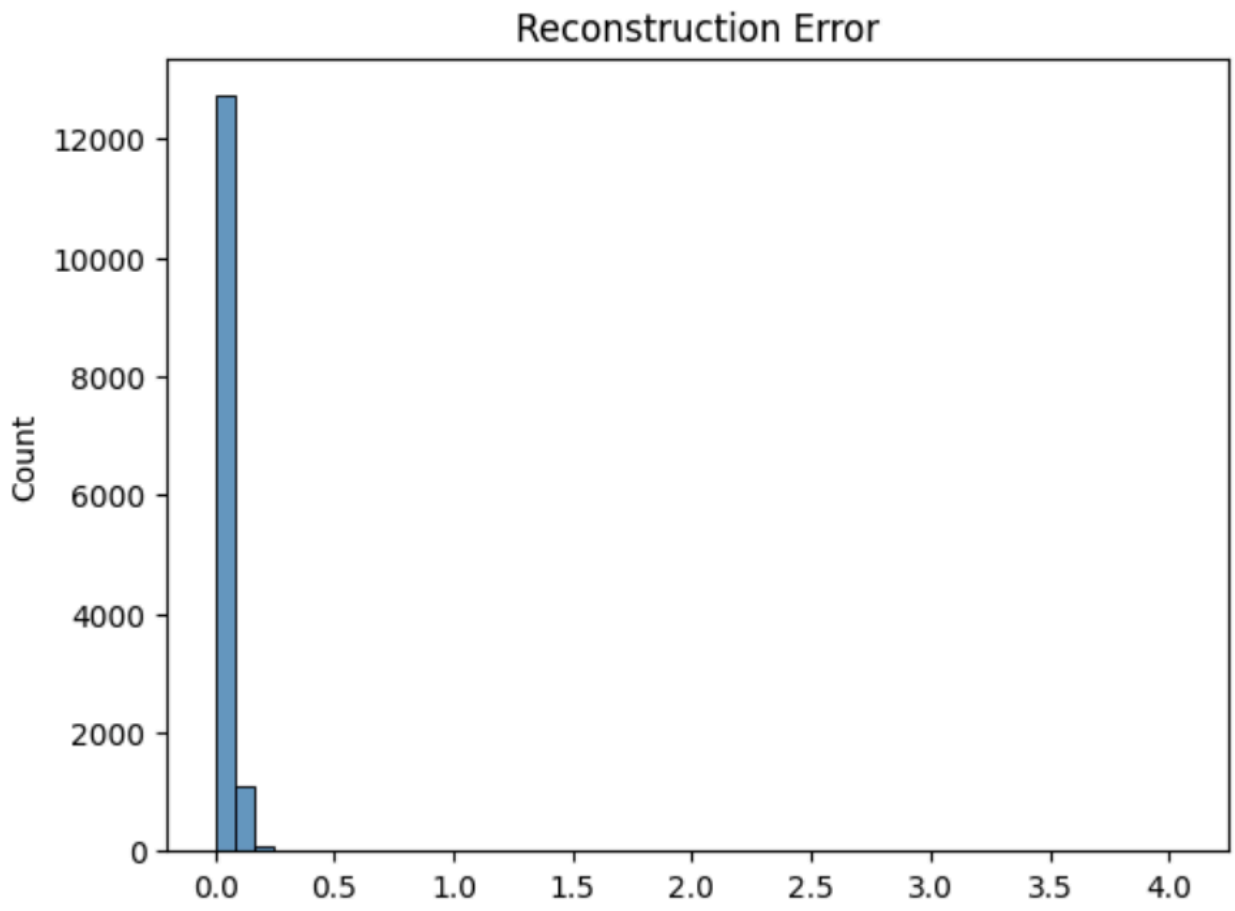
```
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 11)	0
dense (Dense)	(None, 16)	192
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 16)	144
dense_3 (Dense)	(None, 11)	187

```
 Total params: 659 (2.57 KB)
 Trainable params: 659 (2.57 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - loss: 0.4435 - val_loss: 0.2321
Epoch 2/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - loss: 0.1720 - val_loss: 0.1550
Epoch 3/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.1193 - val_loss: 0.1034
Epoch 4/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.0838 - val_loss: 0.0781
Epoch 5/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.0732 - val_loss: 0.0675
Epoch 6/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - loss: 0.0672 - val_loss: 0.0632
Epoch 7/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.0635 - val_loss: 0.0605
Epoch 8/30
1750/1750 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - loss: 0.0610 - val_loss: 0.0598
Epoch 9/30
```


Autoencoder Loss
Train
Validation
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0
5
10
15
20
25
30


Noisy Data
Denoised Output
Original Data



Conclusion

The Autoencoder model was successfully implemented to perform data denoising on the cardiovascular dataset. Although autoencoders are commonly used for image data, they were adapted here for tabular data by introducing synthetic noise and reconstructing the original features. The model demonstrated effective noise reduction and learned meaningful feature representations, making it useful for preprocessing and anomaly detection tasks.